CampusCrate

Campus equipment lending

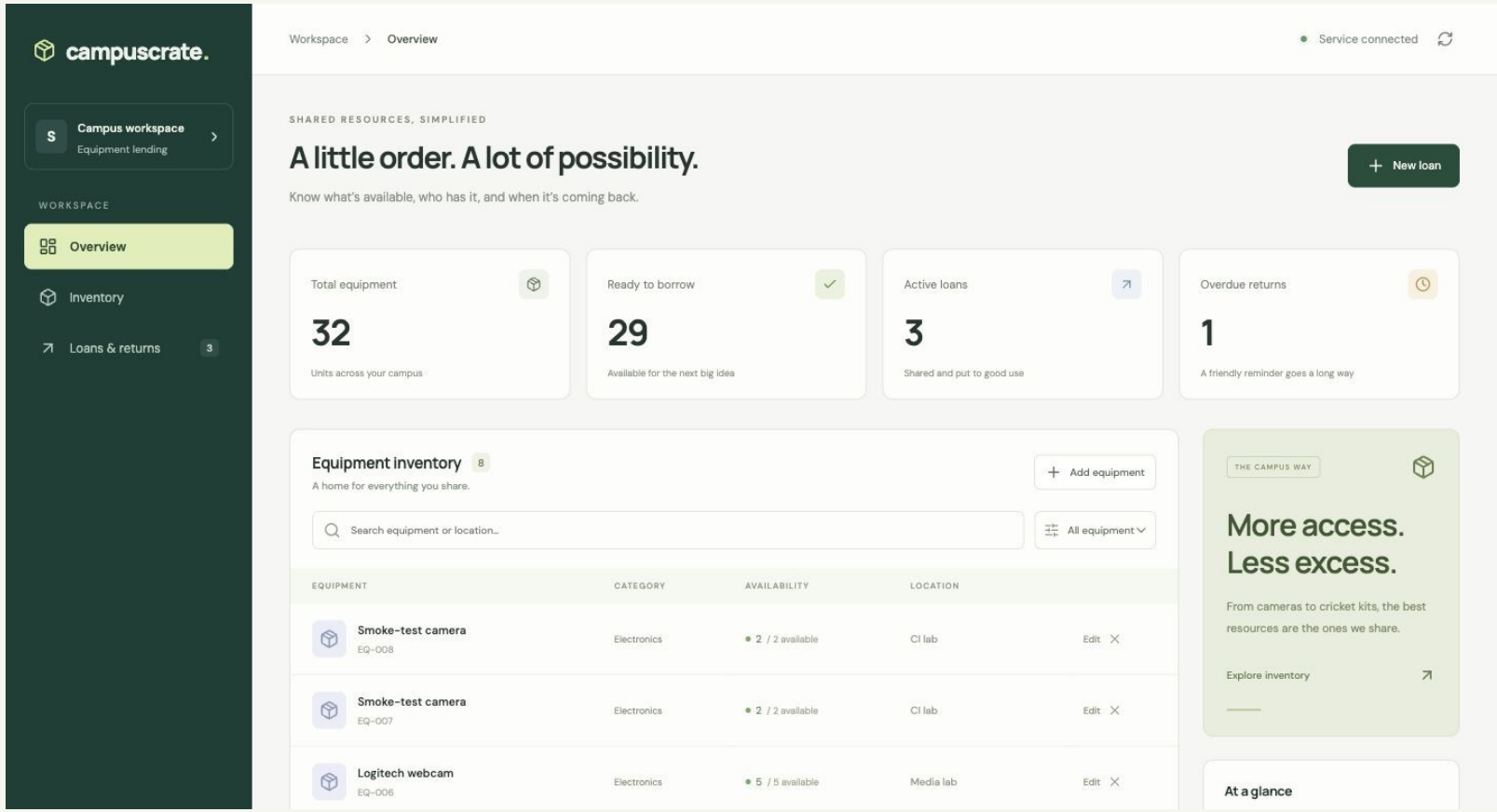
DevOps Final Capstone

Praneeth Budati

praneethb7

Equipment inventory in one workspace

Availability, loans and due dates stay connected



Inventory

Search equipment and filter by category.

Loans and returns

Track who has an item and when it is due.

Recorded Kubernetes view, overview detail

Loan lifecycle and stock protection

The database enforces availability inside a transaction

01 **Borrow**

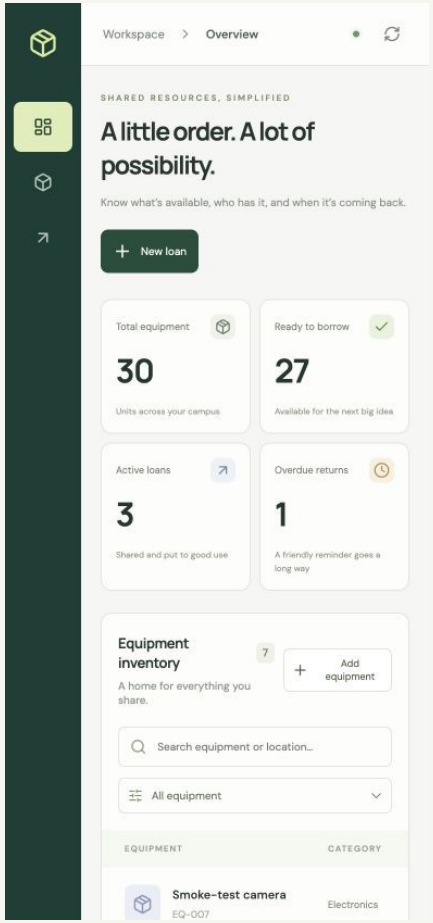
Lock the equipment row, check stock, then record one loan.

02 **Return**

Mark the loan returned once and restore availability.

03 **Preserve history**

Reject duplicate returns and preserve equipment loan history.



Application architecture

Same-origin API routing keeps the browser and backend connected

Layer	Technology	Responsibility
Browser	React and Vite	Inventory and loan interface
Routing	NGINX Ingress	/ to frontend, /api to backend
Application	FastAPI and SQLAlchemy	REST APIs and stock transactions
Persistence	PostgreSQL and Alembic	Loan ledger and versioned schema
Telemetry	Prometheus and Grafana	Scraping, dashboards and alerts

Docker packaging and startup

One Compose command starts the application and its database

docker compose up -d --build

01 **Database becomes healthy**

PostgreSQL passes its readiness check before migration.

02 **Migration completes**

Alembic upgrades the schema before the API starts.

03 **Application becomes ready**

The frontend waits for the backend readiness check.

Tests include database concurrency

Recorded isolated PostgreSQL run

18

tests passed

90.22%

overall statement coverage

Business rules

CRUD, validation, overdue status, loan history and readiness failure

Race conditions

Six borrowers compete for two units. Two requests return the same loan.

GitHub Actions delivery

Source revisions identify the published and deployed images

01 **Validate**

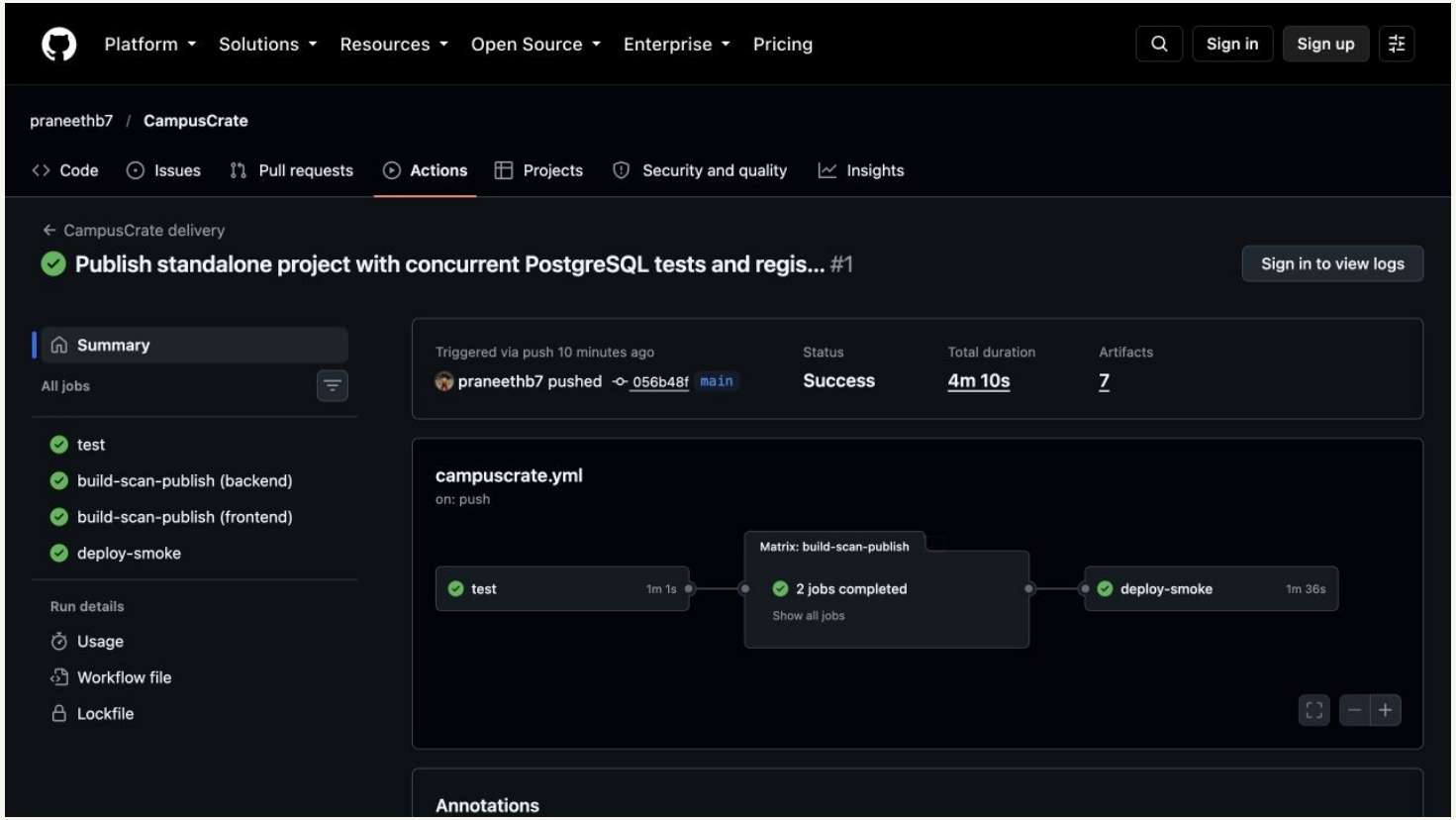
Lint, tests and migration checks

02 **Build and scan**

Both images pass the Trivy gate

03 **Publish and deploy**

GHCR SHA tags, Helm and smoke test



Container security gate

Both recorded image scans passed HIGH and CRITICAL checks

0

HIGH or CRITICAL findings
in the final recorded scans

Remediation

The earlier Debian backend image triggered 44 OS-package findings.

Rebuild on Python Alpine.
Keep the gate and include unfixed CVEs.

Scan results describe the packages and scanner database at that run.

Kubernetes deployment with Helm

The persistent demonstration runs on local Minikube

Component	Deployment behavior
Frontend	Two replicas with an internal ClusterIP Service
Backend	Two to five replicas, CPU autoscaling and probes
Database	PostgreSQL StatefulSet with persistent storage
Migration	Release-specific Alembic Job
Routing	NGINX Ingress for / and /api

Helm rollback restores workload manifests. Database recovery needs its own plan.

Autoscaling under demonstration load

Recorded HPA event confirms scaling from two to five replicas

2

minimum replicas

5

scaled replicas

60%

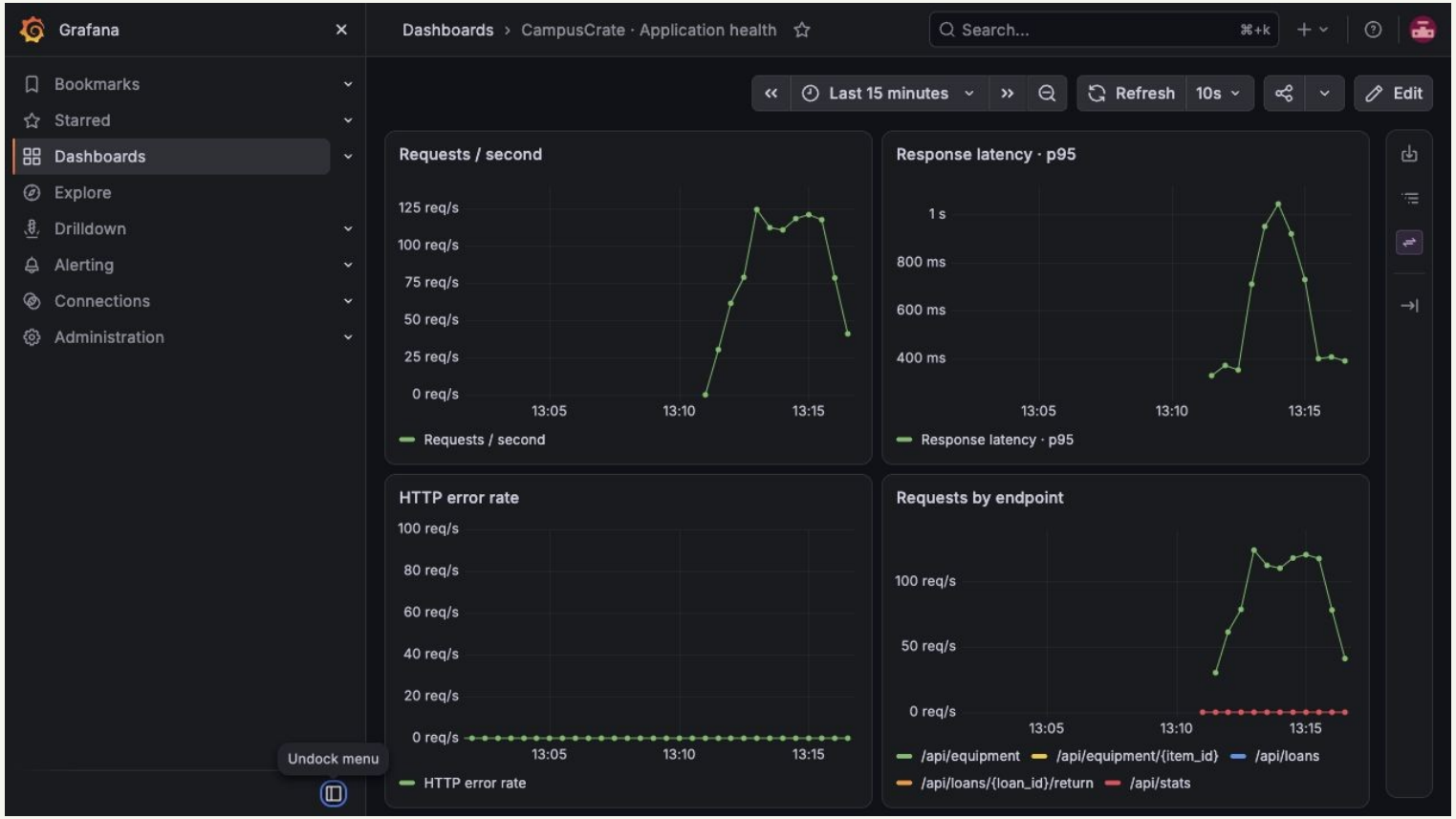
CPU target

SuccessfulRescale: New size: 5

Demo CPU request: 20m. Chart default: 100m.
18,952 successful responses and 4 request errors in 180 seconds.

Application monitoring

Prometheus scrapes the API and Grafana shows the recorded workload



Request rate, p95 latency, HTTP errors and requests by endpoint

AWS infrastructure as code

Terraform validation and mock tests completed. AWS provisioning remains pending

Area	Prepared configuration	Executed verification
Network	VPC, 2 public + 2 private subnets, NAT	Terraform init and validate
Compute	EKS and two managed workers	Mock-provider plan assertions
Access	Restricted API, IAM and OIDC	Configuration and test review
Storage	EBS CSI and encrypted gp3	Configuration and test review

No real AWS plan, apply, Console capture or destroy run.

Troubleshooting and recovery

Separate lab objects keep fault exercises away from the working application

Fault	Diagnostic evidence	Recovery
Missing image tag	Image pull failure in pod events	Correct the image reference
Wrong Service selector	Empty EndpointSlices despite healthy pods	Match selector to pod labels

Probe distinction

A database outage fails readiness with HTTP 503.
Liveness checks the process without depending on the database.

Demonstration sequence

The video will follow the application and its delivery evidence

01 **Equipment workflow**

Create equipment, borrow a unit, return it and inspect the ledger.

02 **Delivery and runtime**

Show Actions, published images, Helm release and running workloads.

03 **Operations**

Explain the HPA event, dashboard, fault diagnosis and AWS scope.

Delivery scope and next steps

The local classroom demonstration has documented production limits

Demonstrated	Further work
Application, tests and local deployment	Add login, authorization and public TLS
CI scans and SHA-tagged images	Add production secrets and backups
Metrics, dashboard and HPA event	Persist monitoring data and tune capacity
Terraform validation and mock tests	Provision, verify and destroy real AWS resources

Video recording remains pending.

CampusCrate

Source code and demonstration evidence

github.com/praneethb7/CampusCrate

Praneeth Budati

Questions